

CS 201, Spring 2026

Homework Assignment 2: Performance Analysis & Optimization

Due: 23:59, March 27, 2026

Introduction

Welcome back to the Bilkent Academy of Wizardry! After successfully implementing the initial Wizard Potion Inventory System, the Headmaster has noticed that as the number of student wizards grows, the system is becoming sluggish. In this assignment, you will evaluate the efficiency of your current implementation and use Large Language Models (LLMs) to identify bottlenecks and optimize your code for high-performance brewing.

Task Descriptions

Task1: Theoretical Analysis

Analyze the time complexity of the following eight functions from your original implementation that is already defined in homework 1 and implemented by you. You must provide the complexity in Big-O and one line justification for each answer:

- `addStudentWizard`
- `removeStudentWizard`
- `brewPotion`
- `discardPotion`
- `transferPotion`
- `showAllStudentWizards`
- `showStudentWizard`
- `showPotion`

Task2: Experimental Profiling

To observe how your code performs under real-world magical demands, you will conduct a series of experiments specifically focused on: `addStudentWizard`, `removeStudentWizard`, `showAllStudentWizards`, `showStudentWizard` functions.

This process begins with test case generation, where you must prepare scenarios with varying values of N (representing the number of operations or students) across at least ten different data points, ranging from 10^2 to 10^5 . Once your test cases are ready, measure the exact

execution time of these functions for each specified value of N. You will then perform visualization by generating graphs that plot N on the x-axis against runtime on the y-axis for each of the functions.

Note: For the `addStudentWizard` and `removeStudentWizard` test cases, make sure that none of the students have any potions and that all `studentName` values are unique.

Note: For the `removeStudentWizard` function, select the element to be removed from somewhere in the middle of the array.

Finally, you are expected to provide a commentary on these graphs, comparing your experimental findings directly with the theoretical complexity analysis you conducted in Task1.

Task3: Bottleneck Identification using LLMs

In this phase, you will collaborate with an LLM (e.g., Gemini, ChatGPT) to improve the performance of your implementation.

1. Provide the LLM with your source code and the graphs generated in Task2.
2. Ask the LLM to identify which functions are creating performance bottlenecks and open to improvement why and how.
3. Document the LLM's reasoning and comment briefly on whether it makes sense.

Note: In this phase, you can interact with the LLMs more than once.

Task4: Optimization

For this task, you are required to enhance the relevant functions by implementing the two mandatory methods provided below. Additionally, you must apply one further optimization recommended by the LLM, which may be implemented in any function you consider appropriate:

1. **Dynamic Array Resizing (Capacity Doubling):** Instead of reallocating and copying the entire array every time a single element is added, implement a strategy where the array capacity doubles when full.

Amortized Analysis Task: For this part, you must include an amortized analysis in your report. This is a method used to show that even though doubling an array is an expensive $O(N)$ task, it happens so rarely that the "average" cost of each addition is actually a constant $O(1)$. Show this phenomena mathematically and refer to the new N vs runtime graphs generated by optimized version of the function.

2. **Swap-with-Last Removal:** To optimize element removal from an array, implement the "swap-with-last" method (replace the element to be deleted with the last element in the array) to achieve $O(1)$ removal, assuming order is not important.

3. **LLM Suggested Optimization:** Implement one specific optimization suggested by the LLM in the Task3. Briefly explain the method, which function it improves and how.

After implementing these improvements, make the complexity analysis and regenerate the N vs runtime graphs to demonstrate the performance improvement.

Note: Even if the function chosen for the LLM-suggested optimization is not included in Task2, you must still prepare both the default (original) and the optimized test cases for that function and compare their performance.

Task5: LLM Interaction Report

You must submit a report detailing your interaction with the LLM. This should include:

- The prompts you used to analyze the code.
- The LLM's suggestions for bottlenecks and optimizations.
- A one paragraph reflection on how helpful (or unhelpful) the LLM was in the debugging/optimization process.

Specifications

1. Your code **MUST NOT** have any memory leaks. Despite using capacity doubling, you must ensure that all allocated memory is properly freed.
2. You are still **PROHIBITED** from using `std::vector`, `std::list`, or other containers from the STL. You must manage the dynamic memory and capacity logic manually.
3. Your graphs should be clear, with axes labeled. You may use any tool (Excel, Python/Matplotlib, etc.) to create them. You should prepare your report using a word processor and convert it into a PDF file. In other words, do not submit images of handwritten answers. Handwritten answers and drawings will not be accepted. In addition, DO NOT submit your report in any other format than pdf.
4. This is an individual assignment for each student. That is, you are NOT allowed to share your work with your classmates.
5. You can use the following code segment to compute the execution time of a code block. For these operations, you need to include the `ctime` header file.

```
//Store the starting time
clock_t startTime = clock();
//Code block
//...
//Compute the number of seconds that passed since the starting time
Clock_t endTime = clock();
double duration = 1000 * double( endTime - startTime ) / CLOCKS_PER_SEC;
cout << "Execution took " << duration << " milliseconds." << endl;
```

Submission Rules

File Format: Submit a zip file named `Firstname-Lastname-StudentID_secX-HW2.zip`.

Contents:

- `WizardPotionInventorySystem.h`, `WizardPotionInventorySystem.cpp` (optimized versions) and `main.cpp` file that contains all the test cases used in your simulations.
- A PDF report containing your answers for all tasks.